

SECTION 1

INTRODUCTION

The MC68030 is a second-generation full 32-bit enhanced microprocessor from Motorola. The MC68030 is a member of the M68000 Family of devices that combines a central processing unit (CPU) core, a data cache, an instruction cache, an enhanced bus controller, and a memory management unit (MMU) in a single VLSI device. The processor is designed to operate at clock speeds beyond 20 MHz. The MC68030 is implemented with 32-bit registers and data paths, 32-bit addresses, a rich instruction set, and versatile addressing modes.

The MC68030 is upward object code compatible with the earlier members of the M68000 Family and has the added features of an on-chip MMU, a data cache, and an improved bus interface. It retains the flexible coprocessor interface pioneered in the MC68020 and provides full IEEE floating-point support through this interface with the MC68881 or MC68882 floating-point coprocessor. Also, the internal functional blocks of this microprocessor are designed to operate in parallel, allowing instruction execution to be overlapped. In addition to instruction execution, the internal caches, the on-chip MMU, and the external bus controller all operate in parallel.

The MC68030 fully supports the nonmultiplexed bus structure of the MC68020, with 32 bits of address and 32 bits of data. The MC68030 bus has an enhanced controller that supports both asynchronous and synchronous bus cycles and burst data transfers. It also supports the MC68020 dynamic bus sizing mechanism that automatically determines device port sizes on a cycle-by-cycle basis as the processor transfers operands to or from external devices.

A block diagram of the MC68030 is shown in Figure 1-1. The instructions and data required by the processor are supplied from the internal caches whenever possible. The MMU translates the logical address generated by the processor into a physical address utilizing its address translation cache (ATC). The bus controller manages the transfer of data between the CPU and memory or devices at the physical address.

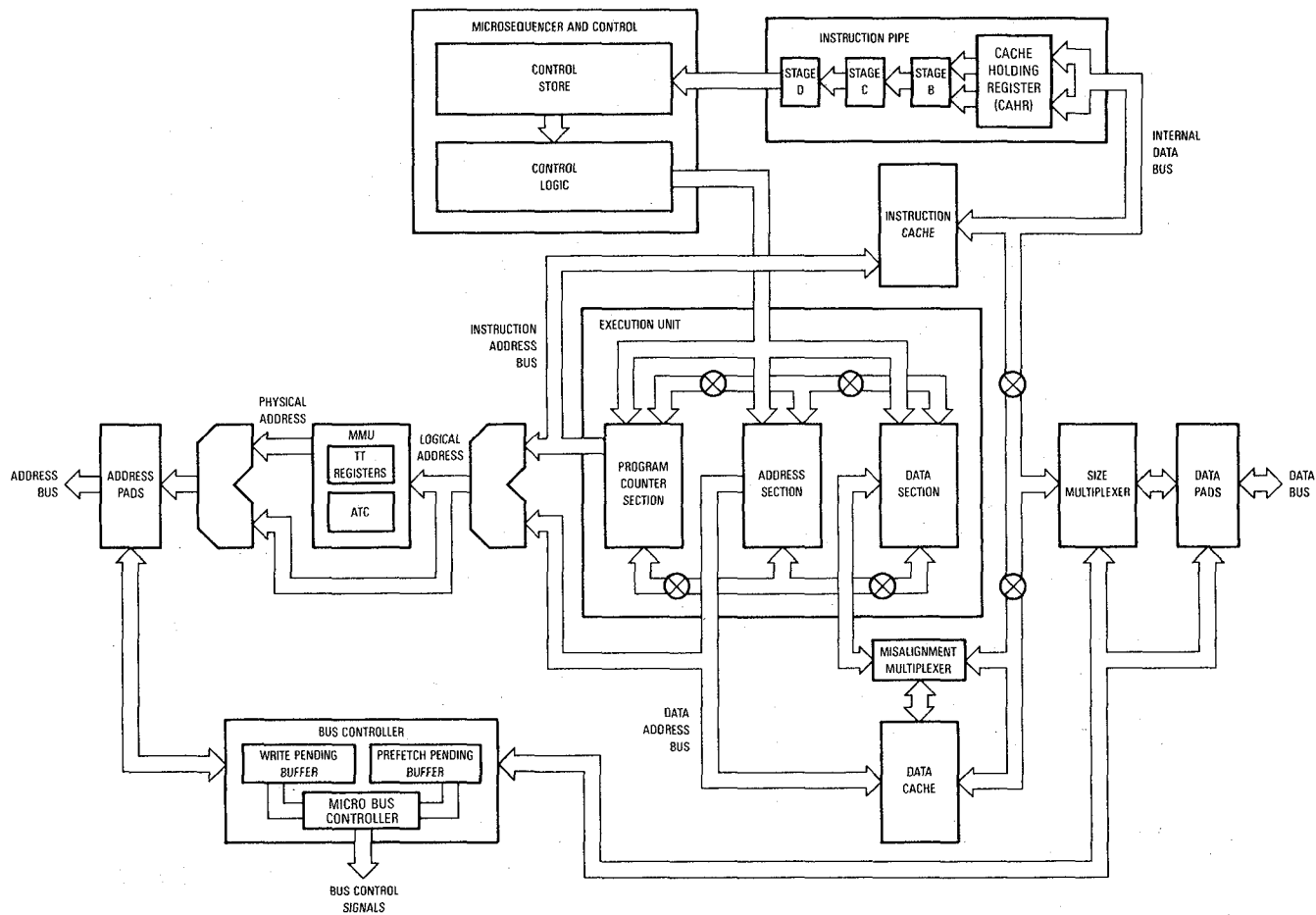


Figure 1-1. Block Diagram

1.1 FEATURES

The features of the MC68030 microprocessor are:

- Object Code Compatible with the MC68020 and Earlier M68000 Microprocessors
- Complete 32-Bit Nonmultiplexed Address and Data Buses
- 16 32-Bit General-Purpose Data and Address Registers
- Two 32-Bit Supervisor Stack Pointers and 10 Special-Purpose Control Registers
- 256-Byte Instruction Cache and 256-Byte Data Cache Can Be Accessed Simultaneously
- Paged MMU that Translates Addresses in Parallel with Instruction Execution and Internal Cache Accesses
- Two Transparent Segments Allow Untranslated Access to Physical Memory To Be Defined for Systems That Transfer Large Blocks of Data between Predefined Physical Addresses — e.g., Graphics Applications
- Pipelined Architecture with Increased Parallelism Allows Accesses to Internal Caches To Occur in Parallel with Bus Transfers and Instruction Execution To Be Overlapped
- Enhanced Bus Controller Supports Asynchronous Bus Cycles (three clocks minimum), Synchronous Bus Cycles (two clocks minimum), and Burst Data Transfers (one clock minimum) all to the Physical Address Space
- Dynamic Bus Sizing Supports 8-, 16-, 32-Bit Memories and Peripherals
- Support for Coprocessors with the M68000 Coprocessor Interface — e.g., Full IEEE Floating-Point Support Provided by the MC68881/MC68882 Floating-Point Coprocessors
- 4-Gbyte Logical and Physical Addressing Range
- Implemented in Motorola's HCMOS Technology That Allows CMOS and HMOS (High-Density NMOS) Gates to be Combined for Maximum Speed, Low Power, and Optimum Die Size
- Processor Speeds Beyond 20 MHz

Both improved performance and increased functionality result from the on-chip implementation of the MMU and the data and instruction caches. The enhanced bus controller and the internal parallelism also provide increased system performance. Finally, the improved bus interface, the reduction in physical size, and the lower power consumption combine to reduce system costs and satisfy cost/performance goals of the system designer.

1.2 MC68030 EXTENSIONS TO THE M68000 FAMILY

In addition to the on-chip instruction cache present in the MC68020, the MC68030 has an internal data cache. Data that is accessed during read cycles may be stored in the on-chip cache, where it is available for subsequent accesses. The data cache reduces the number of external bus cycles when the data operand required by an instruction is already in the data cache.

Performance is enhanced further because the on-chip caches can be internally accessed in a single clock cycle. In addition, the bus controller provides a two-clock cycle synchronous mode and burst mode accesses that can transfer data in as little as one clock per long word.

The MC68030 enhanced microprocessor contains an on-chip MMU that allows address translation to operate in parallel with the CPU core, the internal caches, and the bus controller.

Additional signals support emulation and system analysis. External debug equipment can disable the on-chip caches and the MMU to freeze the MC68030 internal state during breakpoint processing. In addition, the MC68030 indicates:

1. The start of a refill of the instruction pipe
2. Instruction boundaries
3. Pending trace or interrupt processing
4. Exception processing
5. Halt conditions

This status and control information allows external debugging equipment to trace the MC68030 activity and interact nonintrusively with the MC68030 to effectively reduce system debug effort.

1.3 PROGRAMMING MODEL

The programming model of the MC68030 consists of two groups of registers: the user model and the supervisor model. This corresponds to the user and supervisor privilege levels. User programs executing at the user privilege level use the registers of the user model. System software executing at the supervisor level uses the control registers of the supervisor level to perform supervisor functions.

Figure 1-2 shows the user programming model, consisting of 16 32-bit general-purpose registers and two control registers:

- General-Purpose 32-Bit Registers (D0–D7, A0–A7)
- 32-Bit Program Counter (PC)
- 8-Bit Condition Code Register (CCR)

The supervisor programming model consists of the registers available to the user plus 14 control registers:

- Two 32-Bit Supervisor Stack Pointers (ISP and MSP)
- 16-Bit Status Register (SR)
- 32-Bit Vector Base Register (VBR)
- 32-Bit Alternate Function Code Registers (SFC and DFC)
- 32-Bit Cache Control Register (CACR)
- 32-Bit Cache Address Register (CAAR)
- 64-Bit CPU Root Pointer (CRP)
- 64-Bit Supervisor Root Pointer (SRP)
- 32-Bit Translation Control Register (TC)
- 32-Bit Transparent Translation Registers (TT0 and TT1)
- 16-Bit MMU Status Register (MMUSR)

The user programming model remains unchanged from previous M68000 Family microprocessors. The supervisor programming model supplements the user programming model and is used exclusively by the MC68030 system programmers who utilize the supervisor privilege level to implement sensitive operating system functions, I/O control, and memory management subsystems. The supervisor programming model contains all the controls to access and enable the special features of the MC68030. This segregation was carefully planned so that all application software is written to run at the nonprivileged user level and migrates to the MC68030 from any M68000 platform without modification. Since system software is usually modified by system programmers when ported to a new design, the control features are properly placed in the supervisor programming model. For example, the transparent translation feature of the MC68030 is new to the family supervisor programming model for the MC68030 and the two translation registers are

new additions to the family supervisor programming model for the MC68030. Only supervisor code uses this feature, and user application programs remain unaffected.

Registers D0–D7 are used as data registers for bit and bit field (1 to 32 bits), byte (8 bit), word (16 bit), long-word (32 bit), and quad-word (64 bit) operations. Registers A0–A6 and the user, interrupt, and master stack pointers are address registers that may be used as software stack pointers or base address registers. Register A7 (shown as A7' and A7'' in Figure 1-3) is a register designation that applies to the user stack pointer in the user privilege level and to either the interrupt or master stack pointer in the supervisor privilege level. In the supervisor privilege level, the active stack pointer (interrupt or master) is called the supervisor stack pointer (SSP). In addition,

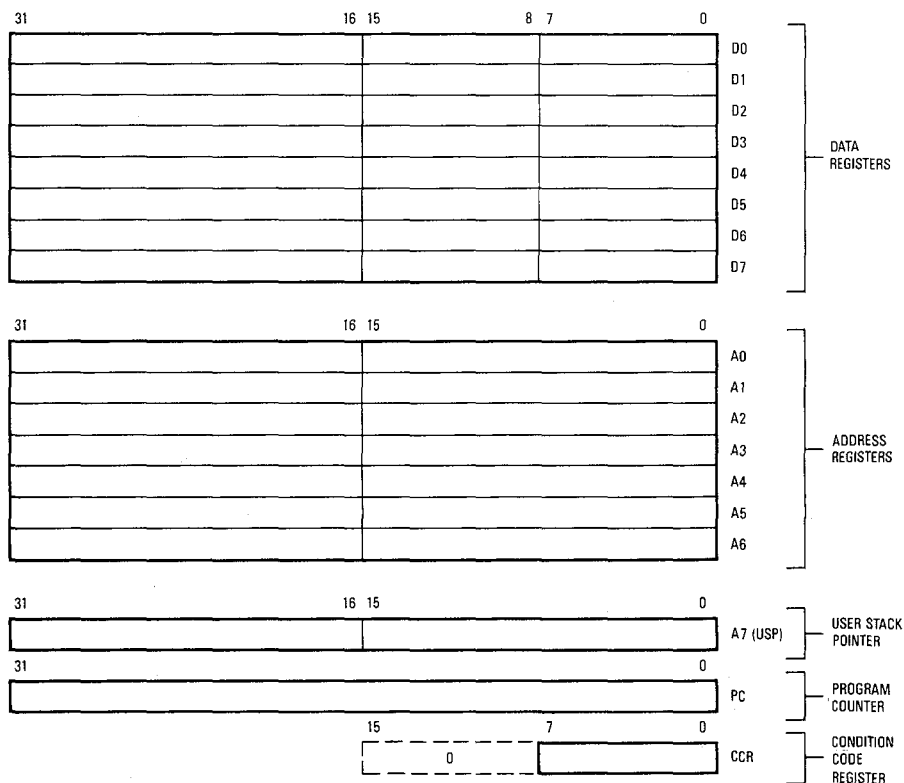


Figure 1-2. User Programming Model

the address registers may be used for word and long-word operations. All of the 16 general-purpose registers (D0–D7, A0–A7) may be used as index registers.

The program counter (PC) contains the address of the next instruction to be executed by the MC68030. During instruction execution and exception processing, the processor automatically increments the contents of the PC or places a new value in the PC, as appropriate.

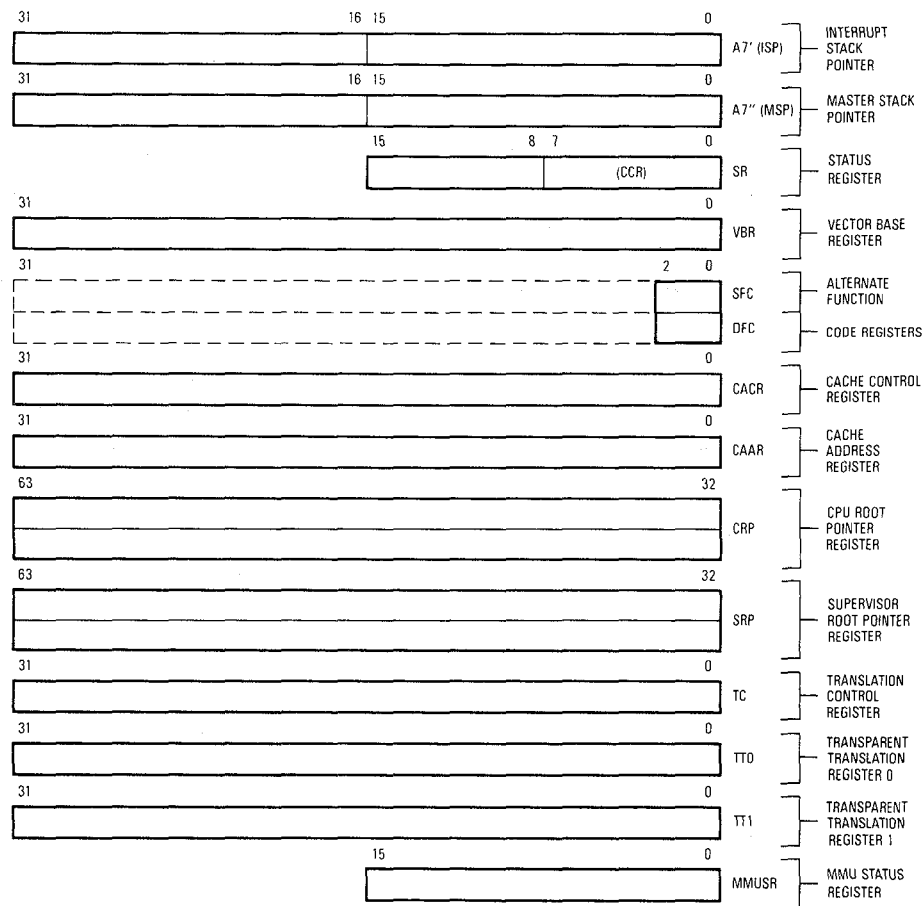


Figure 1-3. Supervisor Programming Model Supplement

The status register, SR, (see Figure 1-4) stores the processor status. It contains the condition codes that reflect the results of a previous operation and can be used for conditional instruction execution in a program. The condition codes are extend (X), negative (N), zero (Z), overflow (V), and carry (C). The user byte containing the condition codes is the only portion of the status register information available in the user privilege level, and it is referenced as the CCR in user programs. In the supervisor privilege level, software can access the full status register, including the interrupt priority mask (three bits) as well as additional control bits. These bits indicate whether the processor is in:

1. One of two trace modes (T1, T0)
2. Supervisor or user privilege level (S)
3. Master or interrupt mode (M)

The vector base register (VBR) contains the base address of the exception vector table in memory. The displacement of an exception vector is added to the value in this register to access the vector table.

Alternate function code registers, SFC and DFC, contain 3-bit function codes. Function codes can be considered extensions of the 32-bit linear address that optionally provide as many as eight 4-Gbyte address spaces. Function codes are automatically generated by the processor to select address spaces for data and program at the user and supervisor privilege levels and a CPU address space for processor functions (e.g., coprocessor communications). Registers SFC and DFC are used by certain instructions to explicitly specify the function codes for operations.

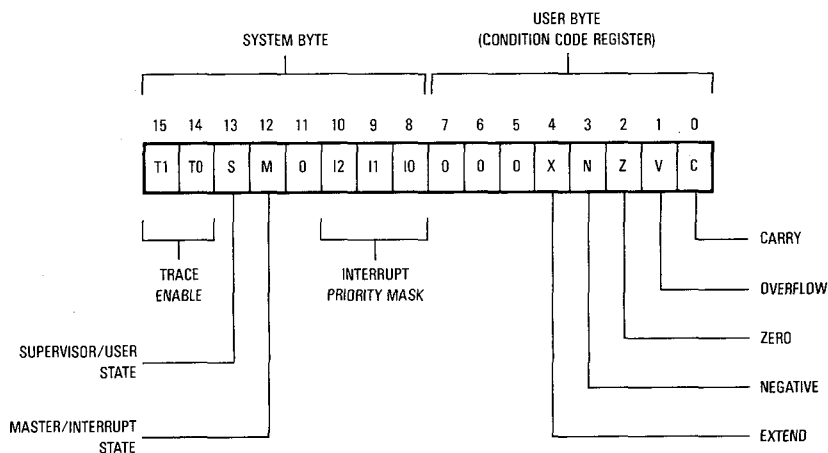


Figure 1-4. Status Register

The cache control register (CACR) controls the on-chip instruction and data caches of the MC68030. The cache address register (CAAR) stores an address for cache control functions.

The CPU root pointer (CRP) contains a pointer to the root of the translation tree for the currently executing task of the MC68030. This tree contains the mapping information for the task's address space. When the MC68030 is configured to provide a separate address space for supervisor routines, the supervisor root pointer (SRP) contains a pointer to the root of the translation tree describing the supervisor's address space.

The translation control register (TC) consists of several fields that control address translation. These fields enable and disable address translation, enable and disable the use of SRP for the supervisor address space, and select or ignore the function codes in translating addresses. Other fields define the size of memory pages, the number of address bits used in translation, and the translation table structure.

The transparent translation registers, TT0 and TT1, can each specify separate blocks of memory as directly accessible without address translation. Logical addresses in these areas become the physical addresses for memory access. Function codes and the eight most significant bits of the address can be used to define the area of memory and type of access; either read, write, or both types of memory access can be directly mapped. The transparent translation feature allows rapid movement of large blocks of data in memory or I/O space without disturbing the context of the on-chip address translation cache or incurring delays associated with translation table lookups. This feature is useful to graphics, controller, and real-time applications.

The MMU status register (MMUSR) contains memory management status information resulting from a search of the address translation cache or the translation tree for a particular logical address.

1.4 DATA TYPES AND ADDRESSING MODES

Seven basic data types are supported:

1. Bits
2. Bit Fields (Fields of consecutive bits, 1–32 bits long)
3. BCD Digits (Packed: 2 digits byte, Unpacked: 1 digit/byte)
4. Byte Integers (8 bits)
5. Word Integers (16 bits)
6. Long-Word Integers (32 bits)
7. Quad-Word Integers (64 bits)

In addition, the instruction set supports operations on other data types such as memory addresses. The coprocessor mechanism allows direct support of floating-point operations with the MC68881 and MC68882 floating-point coprocessors as well as specialized user-defined data types and functions.

The 18 addressing modes, shown in Table 1-1, include nine basic types:

1. Register Direct
2. Register Indirect
3. Register Indirect with Index
4. Memory Indirect
5. Program Counter Indirect with Displacement
6. Program Counter Indirect with Index
7. Program Counter Memory Indirect
8. Absolute
9. Immediate

The register indirect addressing modes can also postincrement, predecrement, offset, and index addresses. The program counter relative mode also has index and offset capabilities. As in the MC68020, both modes are extended to provide indirect reference through memory. In addition to these addressing modes, many instructions implicitly specify the use of the condition code register, stack pointer, and/or program counter.

1.5 INSTRUCTION SET OVERVIEW

The instructions in the MC68030 instruction set are listed in Table 1-2. The instruction set has been tailored to support structured high-level languages and sophisticated operating systems. Many instructions operate on bytes, words, or long words, and most instructions can use any of the 18 addressing modes.

Table 1-1. Addressing Modes

Addressing Modes	Syntax
Register Direct Data Register Direct Address Register Direct	Dn An
Register Indirect Address Register Indirect Address Register Indirect with Postincrement Address Register Indirect with Predecrement Address Register Indirect with Displacement	(An) (An) + – (An) (d ₁₆ ,An)
Register Indirect with Index Address Register Indirect with Index (8-Bit Displacement) Address Register Indirect with Index (Base Displacement)	(dg,An,Xn) (bd,An,Xn)
Memory Indirect Memory Indirect Postindexed Memory Indirect Preindexed	([bd,An],Xn,od) ([bd,An,Xn],od)
Program Counter Indirect with Displacement	(d ₁₆ ,PC)
Program Counter Indirect with Index PC Indirect with Index (8-Bit Displacement) PC Indirect with Index (Base Displacement)	(dg,PC,Xn) (bd,PC,Xn)
Program Counter Memory Indirect PC Memory Indirect Postindexed PC Memory Indirect Preindexed	([bd,PC],Xn,od) ([bd,PC,Xn],od)
Absolute Absolute Short Absolute Long	(xxx).W (xxx).L
Immediate	#(data)

NOTES:

- Dn = Data Register, D0–D7
- An = Address Register, A0–A7
- 8, d₁₆ = A two's-complement or sign-extended displacement; added as part of the effective address calculation; size is 8 (dg) or 16 (d₁₆) bits; when omitted, assemblers use a value of zero.
- Xn = Address or data register used as an index register; form is Xn.SIZE*SCALE, where SIZE is .W or .L (indicates index register size) and SCALE is 1, 2, 4, or 8 (index register is multiplied by SCALE); use of SIZE and/or SCALE is optional.
- bd = A two's-complement base displacement; when present, size can be 16 or 32 bits.
- od = Outer displacement, added as part of effective address calculation after any memory indirection; use is optional with a size of 16 or 32 bits.
- PC = Program Counter
- (data) = Immediate value of 8, 16, or 32 bits
- () = Effective Address
- [] = Use as indirect access to long-word address.

1.6 VIRTUAL MEMORY AND VIRTUAL MACHINE CONCEPTS

The full addressing range of the MC68030 is 4 Gbytes (4,294,967,296 bytes) in each of eight address spaces. Even though most systems implement a smaller physical memory, the system can be made to appear to have a full 4 Gbytes of memory available to each user program by using virtual memory techniques.

In a virtual memory system, a user program can be written as if it has a large amount of memory available, when the physical memory actually present is much smaller. Similarly, a system can be designed to allow user programs to access devices that are not physically present in the system, such as tape drives, disk drives, printers, terminals, and so forth. With proper software emulation, a physical system can appear to be any other M68000 computer system to a user program, and the program can be given full access to all of the resources of that emulated system. Such an emulated system is called a *virtual machine*.

1.6.1 Virtual Memory

A system that supports virtual memory has a *limited* amount of high-speed physical memory that can be accessed directly by the processor and maintains an image of a much larger virtual memory on a secondary storage device such as a large-capacity disk drive. When the processor attempts to access a location in the virtual memory map that is not resident in physical memory, a page fault occurs. The access to that location is temporarily suspended while the necessary data is fetched from secondary storage and placed in physical memory. The suspended access is then either restarted or continued.

The MC68030 uses instruction continuation to support virtual memory. When a bus cycle is terminated with a bus error, the microprocessor suspends the current instruction and executes the virtual memory bus error handler. When the bus error handler has completed execution, it returns control to the program that was executing when the error was detected, reruns the faulted bus cycle (when required), and continues the suspended instruction.

Table 1-2. Instruction Set

Mnemonic	Description	Mnemonic	Description
ABCD	Add Decimal with Extend	MOVE USP	Move User Stack Pointer
ADD	Add	MOVEC	Move Control Register
ADDA	Add Address	MOVEM	Move Multiple Registers
ADDI	Add Immediate	MOVEP	Move Peripheral
ADDQ	Add Quick	MOVEQ	Move Quick
ADDX	Add with Extend	MOVES	Move Alternate Address Space
AND	Logical AND		
ANDI	Logical AND Immediate	MULS	Signed Multiply
ASL, ASR	Arithmetic Shift Left and Right	MULU	Unsigned Multiply
Bcc	Branch Conditionally	NBCD	Negate Decimal with Extend
BCHG	Test Bit and Change	NEG	Negate
BCLR	Test Bit and Clear	NEGX	Negate with Extend
BFCHG	Test Bit Field and Change	NOP	No Operation
BFCLR	Test Bit Field and Clear	NOT	Logical Complement
BFEXTS	Signed Bit Field Extract	OR	Logical Inclusive OR
BFEXTU	Unsigned Bit Field Extract	ORI	Logical Inclusive OR Immediate
BFFFO	Bit Field Find First One	ORI CCR	Logical Inclusive OR Immediate to Condition Codes
BFINS	Bit Field Insert	ORI SR	Logical Inclusive OR Immediate to Status Register
BFSET	Test Bit Field and Set		
BFTST	Test Bit Field	PACK	Pack BCD
BKPT	Breakpoint	PEA	Push Effective Address
BRA	Branch		
BSET	Test Bit and Set	PFLUSH	Flush Entry(ies) in the ATC
BSR	Branch to Subroutine	PFLUSHA	Flush All Entries in the ATC
BTST	Test Bit	PLOADR,	Load Entry into the ATC
		PLOADW	
CAS	Compare and Swap Operands	PMOVE	Move to/from MMU Registers
CAS2	Compare and Swap Dual Operands	PMOVEFD	Move to/from MMU Registers with Flush Disable
CHK	Check Register Against Bound	PTESTR,	Test a Logical Address
CHK2	Check Register Against Upper and Lower Bounds	PTESTW	
CLR	Clear	RESET	Reset External Devices
CMP	Compare	ROL, ROR	Rotate Left and Right
CMPA	Compare Address	ROXL, ROXR	Rotate with Extend Left and Right
CMPI	Compare Immediate	RTD	Return and Deallocate
CMPM	Compare Memory to Memory	RTE	Return from Exception
CMP2	Compare Register Against Upper and Lower Bounds	RTR	Return and Restore Codes
		RTS	Return from Subroutine
DBcc	Test Condition, Decrement and Branch		
DIVS, DIVSL	Signed Divide	SB CD	Subtract Decimal with Extend
DIVU, DIVUL	Unsigned Divide	Scc	Set Conditionally
EOR	Logical Exclusive OR	STOP	Stop
EORI	Logical Exclusive OR Immediate	SUB	Subtract
EXG	Exchange Registers	SUBA	Subtract Address
EXT, EXTB	Sign Extend	SUBI	Subtract Immediate
ILLEGAL	Take Illegal Instruction Trap	SUBQ	Subtract Quick
		SUBX	Subtract with Extend
JMP	Jump	SWAP	Swap Register Words
JSR	Jump to Subroutine		
LEA	Load Effective Address	TAS	Test Operand and Set
LINK	Link and Allocate	TRAP	Trap
LSL, LSR	Logical Shift Left and Right	TRAPcc	Trap Conditionally
		TRAPV	Trap on Overflow
MOVE	Move	TST	Test Operand
MOVEA	Move Address		
MOVE CCR	Move Condition Code Register	UNLK	Unlink
MOVE SR	Move Status Register	UNPK	Unpack BCD

Coprocessor Instructions

1

Mnemonic	Description
cpBcc	Branch Conditionally
cpDBcc	Test Coprocessor Condition, Decrement and Branch
cpGEN	Coprocessor General Instruction

Mnemonic	Description
cpRESTORE	Restore Internal State of Coprocessor
cpSAVE	Save Internal State of Coprocessor
cpScc	Set Conditionally
cpTRAPcc	Trap Conditionally

1.6.2 Virtual Machine

A typical use for a virtual machine system is the development of software, such as an operating system, for a new machine also under development and not yet available for programming use. In a virtual machine system, a governing operating system emulates the hardware of the new machine and allows the new software to be executed and debugged as though it were running on the new hardware. Since the new software is controlled by the governing operating system, it is executed at a lower privilege level than the governing operating system. Thus, any attempts by the new software to use virtual resources that are not physically present (and should be emulated) are trapped to the governing operating system and performed by its software.

In the MC68030 implementation of a virtual machine, the virtual application runs at the user privilege level. The governing operating system executes at the supervisor privilege level and any attempt by the new operating system to access supervisor resources or execute privileged instructions causes a trap to the governing operating system.

Instruction continuation is used to support virtual I/O devices in memory-mapped input/output systems. Control and data registers for the virtual device are simulated in the memory map. An access to a virtual register causes a fault and the function of the register is emulated by software.

1.7 THE MEMORY MANAGEMENT UNIT

The MMU supports virtual memory systems by translating logical addresses to physical addresses using translation tables stored in memory. The MMU stores address mappings in an address translation cache (ATC) that contains the most recently used translations. When the ATC contains the address for a bus cycle requested by the CPU, a translation table search is not performed. Features of the MMU include:

- Multiple Level Translation Tables with Short- and Long-Format Descriptors for Efficient Table Space Usage
- Table Searches Automatically Performed in Microcode
- 22-Entry Fully Associative ATC
- Address Translations and Internal Instruction and Data Cache Accesses Performed in Parallel
- Eight Page Sizes Available Ranging from 256 to 32K Bytes
- Two Optional Transparent Blocks
- User and Supervisor Root Pointer Registers
- Write Protection and Supervisor Protection Attributes
- Translations Enabled/Disabled by Software
- Translations Can Be Disabled with External $\overline{\text{MMUDIS}}$ Signal
- Used and Modified Bits Automatically Maintained in Tables and ATC
- Cache Inhibit Output ($\overline{\text{CIOUT}}$) Signal Can Be Asserted on a Page-by-Page Basis
- 32-Bit Internal Logical Address with Capability To Ignore as many as 15 Upper Address Bits
- 3-Bit Function Code Supports Separate Address Spaces
- 32-Bit Physical Address

The memory management function performed by the MMU is called demand paged memory management. Since a task specifies the areas of memory it requires as it executes, memory allocation is supported on a demand basis. If a requested access to memory is not currently mapped by the system, then the access causes a demand for the operating system to load or allocate the required memory image. The technique used by the MC68030 is paged memory management because physical memory is managed in blocks of a specified number of bytes, called page frames. The logical address space is divided

into fixed-size pages that contain the same number of bytes as the page frames. Memory management assigns a physical base address to a logical page. The system software then transfers data between secondary storage and memory one or more pages at a time.

1.8 PIPELINED ARCHITECTURE

The MC68030 uses a three-stage pipelined internal architecture to provide for optimum instruction throughput. The pipeline allows as many as three words of a single instruction or three consecutive instructions to be decoded concurrently.

1.9 THE CACHE MEMORIES

Due to locality of reference, instructions and data that are used in a program have a high probability of being reused within a short time. Additionally, instructions and data operands that reside in proximity to the instructions and data currently in use also have a high probability of being utilized within a short period. To exploit these locality characteristics, the MC68030 contains two on-chip logical caches, a data cache, and an instruction cache.

Each of the caches stores 256 bytes of information, organized as 16 entries, each containing a block of four long words (16 bytes). The processor fills the cache entries either one long word at a time or, during burst mode accesses, four long words consecutively. The burst mode of operation not only fills the cache efficiently but also captures adjacent instruction or data items that are likely to be required in the near future due to locality characteristics of the executing task.

The caches improve the overall performance of the system by reducing the number of bus cycles required by the processor to fetch information from memory and by increasing the bus bandwidth available for other bus masters in the system. Addition of the data cache in the MC68030 extends the benefits of cache techniques to all memory accesses. During a write cycle, the data cache circuitry writes data to a cached data item as well as to the item in memory, maintaining consistency between data in the cache and that in memory. However, writing data that is not in the cache may or may not cause the data item to be stored in the cache, depending on the write allocation policy selected in the cache control register (CACR).